

Lecture 8:

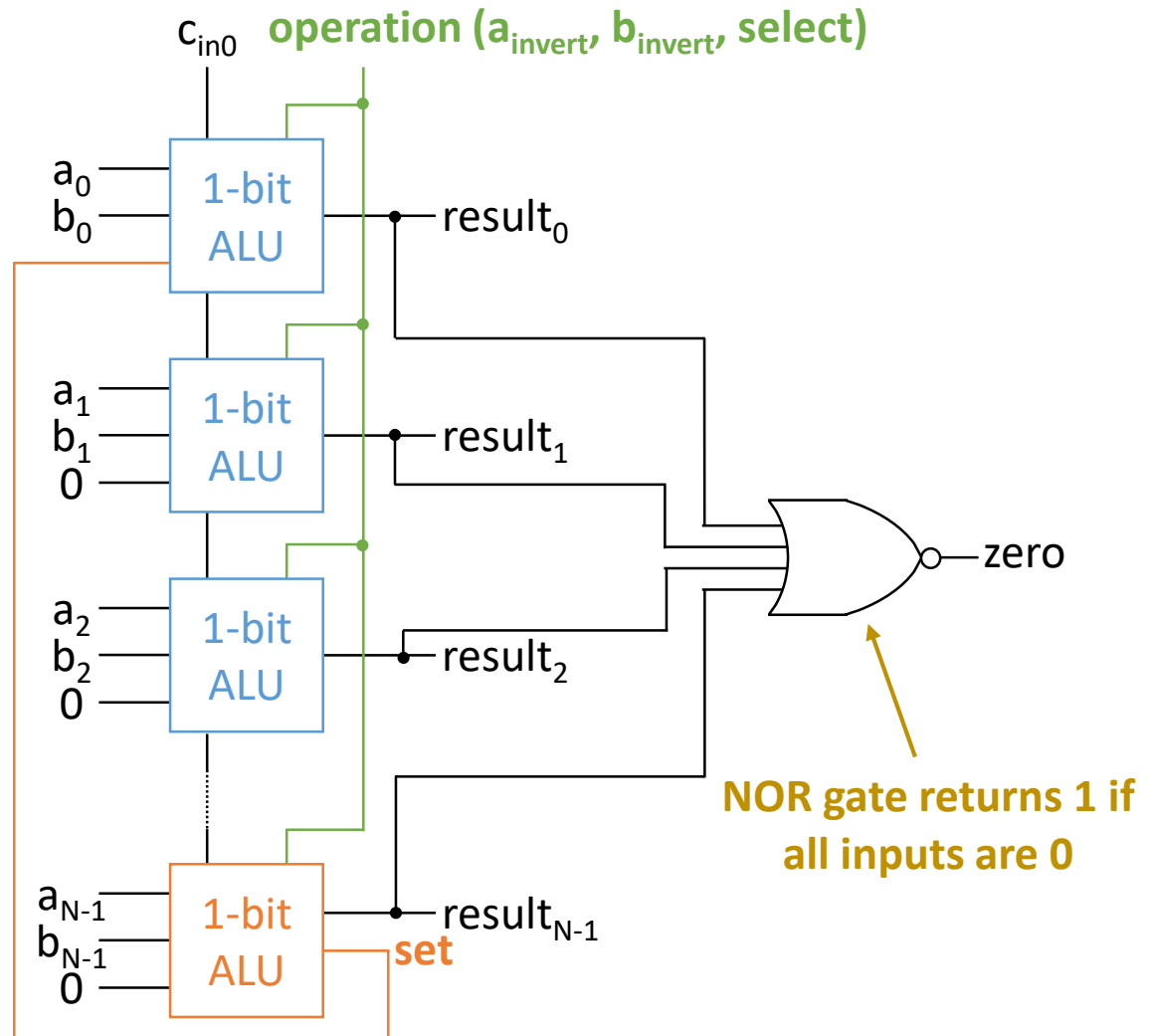
Building an ALU – Part 2

CMPS 221 – Computer Organization and Design

Equality

- An equality check sets a flag to 1 if $a = b$ and 0 otherwise
 - Used by instructions that branch to different parts of the code based on some condition (ignore ALU output)
- Equality can be implemented by subtracting the two operands and checking if the result is zero
$$a = b \Leftrightarrow a - b = 0$$
- The result is zero if all its bits are zero

N-bit ALU with Equality



Overflow

- When using a fixed number of bits, there is a limited range of values that can be represented
 - Range for **unsigned** N-bit numbers: 0 to 2^N-1
 - Range for **signed** N-bit numbers: -2^{N-1} to $2^{N-1}-1$

- e.g., range of signed 3-bit numbers
- | | |
|-----|-----------------|
| 011 | $3 = 2^{3-1}-1$ |
| 010 | |
| 001 | |
| 000 | |
| 111 | |
| 110 | |
| 101 | |
| 100 | $-4 = -2^{3-1}$ |

- If the sum of two numbers is outside this range, an overflow occurs and the result is incorrect
 - e.g., for signed 3-bit numbers: $010+011=101$ ($2+3=-3$)

Overflow Detection

operation	a	b	sum	overflow
Add	+	+	+	No
Add	+	+	-	Yes
Add	+	-	+	No
Add	+	-	-	No
Add	-	+	+	No
Add	-	+	-	No
Add	-	-	+	Yes
Add	-	-	-	No
Sub/SLT	+	+	+	No
Sub/SLT	+	+	-	No
Sub/SLT	+	-	+	No
Sub/SLT	+	-	-	Yes
Sub/SLT	-	+	+	Yes
Sub/SLT	-	+	-	No
Sub/SLT	-	-	+	No
Sub/SLT	-	-	-	No

take a and b after mux
(denote as a' and b',
b' is flipped on sub/slt)

Overflow Detection



operation	a'	b'	sum	overflow
Add	+	+	+	No
Add	+	+	-	Yes
Add	+	-	+	No
Add	+	-	-	No
Add	-	+	+	No
Add	-	+	-	No
Add	-	-	+	Yes
Add	-	-	-	No
Sub/SLT	+	-	+	No
Sub/SLT	+	-	-	No
Sub/SLT	+	+	+	No
Sub/SLT	+	+	-	Yes
Sub/SLT	-	-	+	Yes
Sub/SLT	-	-	-	No
Sub/SLT	-	+	+	No
Sub/SLT	-	+	-	No



conditions for
add and sub/slt
are the same

Overflow Detection

a'	b'	sum	overflow
+	+	+	No
+	+	-	Yes
+	-	+	No
+	-	-	No
-	+	+	No
-	+	-	No
-	-	+	Yes
-	-	-	No

only need most
significant bit to find
out if number is
positive or negative

Overflow Detection



a'_{N-1}	b'_{N-1}	sum_{N-1}	overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

$$overflow = \overline{a'_{N-1}} \cdot \overline{b'_{N-1}} \cdot sum_{N-1} + a'_{N-1} \cdot b'_{N-1} \cdot \overline{sum_{N-1}}$$

only need most
significant bit to find
out if number is
positive or negative

Overflow Detection



a'_{N-1}	b'_{N-1}	sum_{N-1}	overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

$$overflow = \overline{a'_{N-1}} \cdot \overline{b'_{N-1}} \cdot sum_{N-1} + a'_{N-1} \cdot b'_{N-1} \cdot \overline{sum_{N-1}}$$

11 gate inputs

Can we do better?

Overflow Detection

a'_{N-1}	b'_{N-1}	sum_{N-1}	overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

$$overflow = \overline{a'_{N-1}} \cdot \overline{b'_{N-1}} \cdot sum_{N-1} + a'_{N-1} \cdot b'_{N-1} \cdot \overline{sum_{N-1}}$$

Express overflow in terms of c_{inN-1} and c_{outN-1} :

c_{inN-1}	c_{outN-1}	overflow
0	0	0
0	1	1
1	0	1
1	1	0

$$overflow = \overline{c_{inN-1}} \cdot c_{outN-1} + c_{inN-1} \cdot \overline{c_{outN-1}}$$

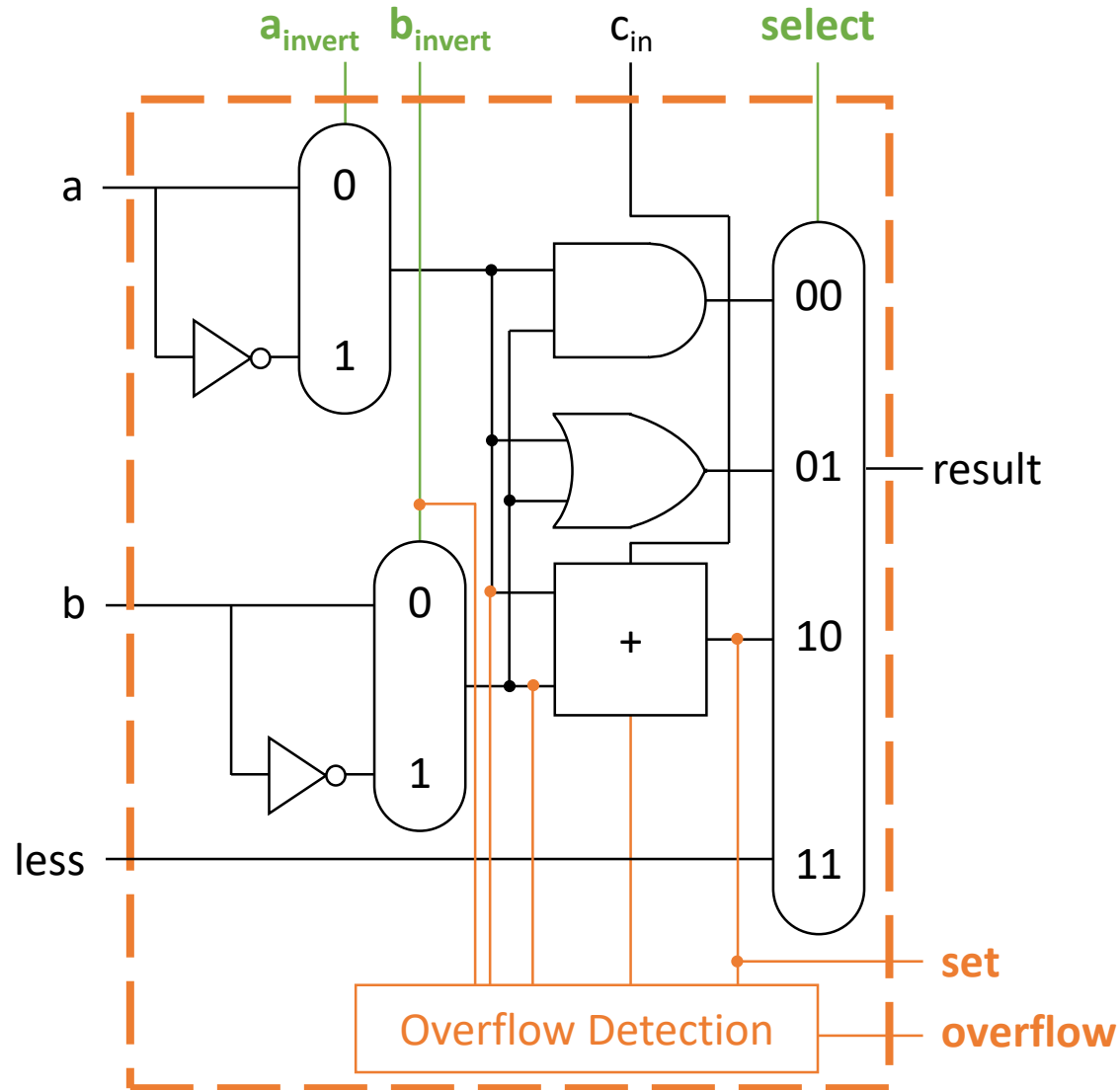
Recall truth table for 1-bit adder:

Inputs			Outputs	
a	b	c_{in}	c_{out}	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

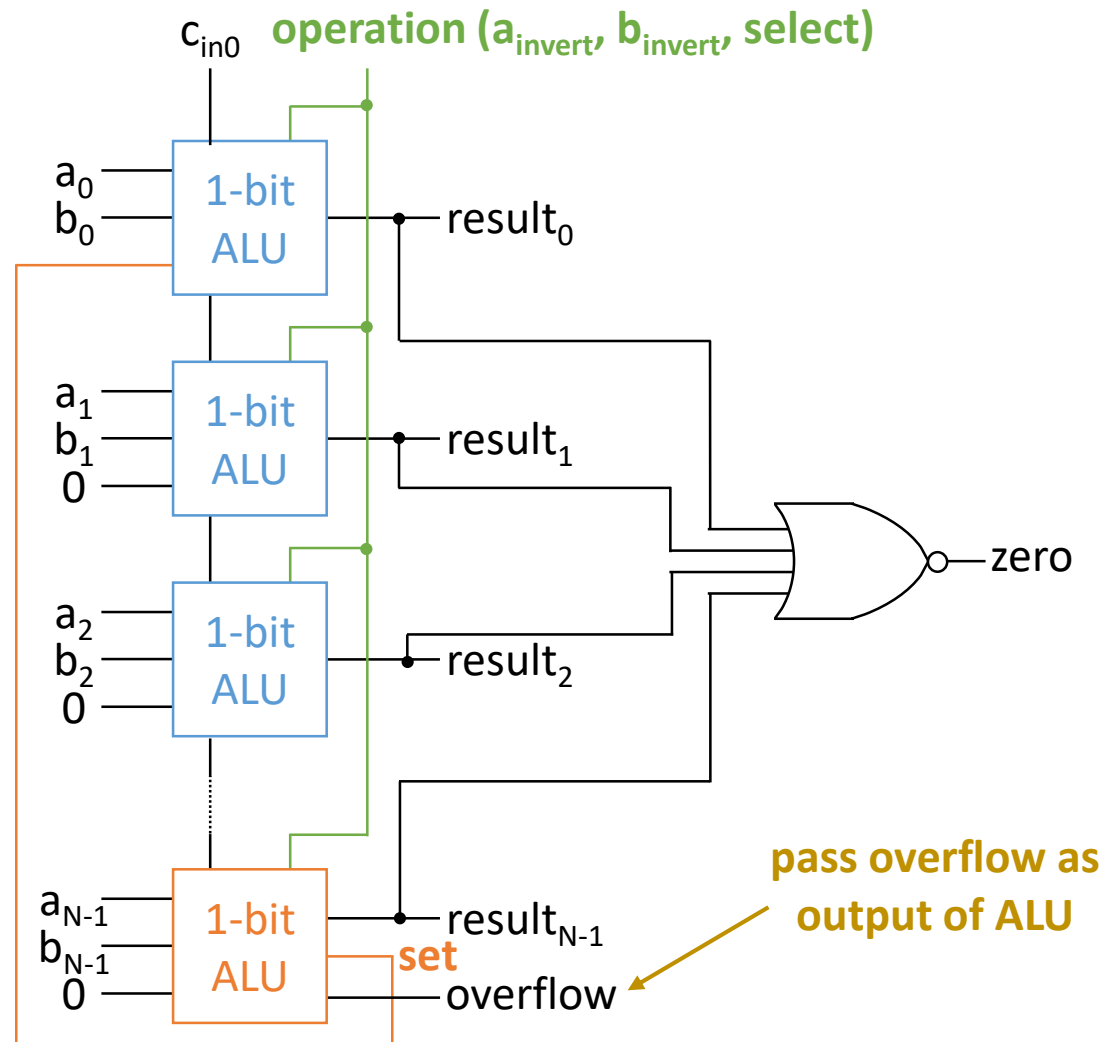
11 gate inputs

8 gate inputs

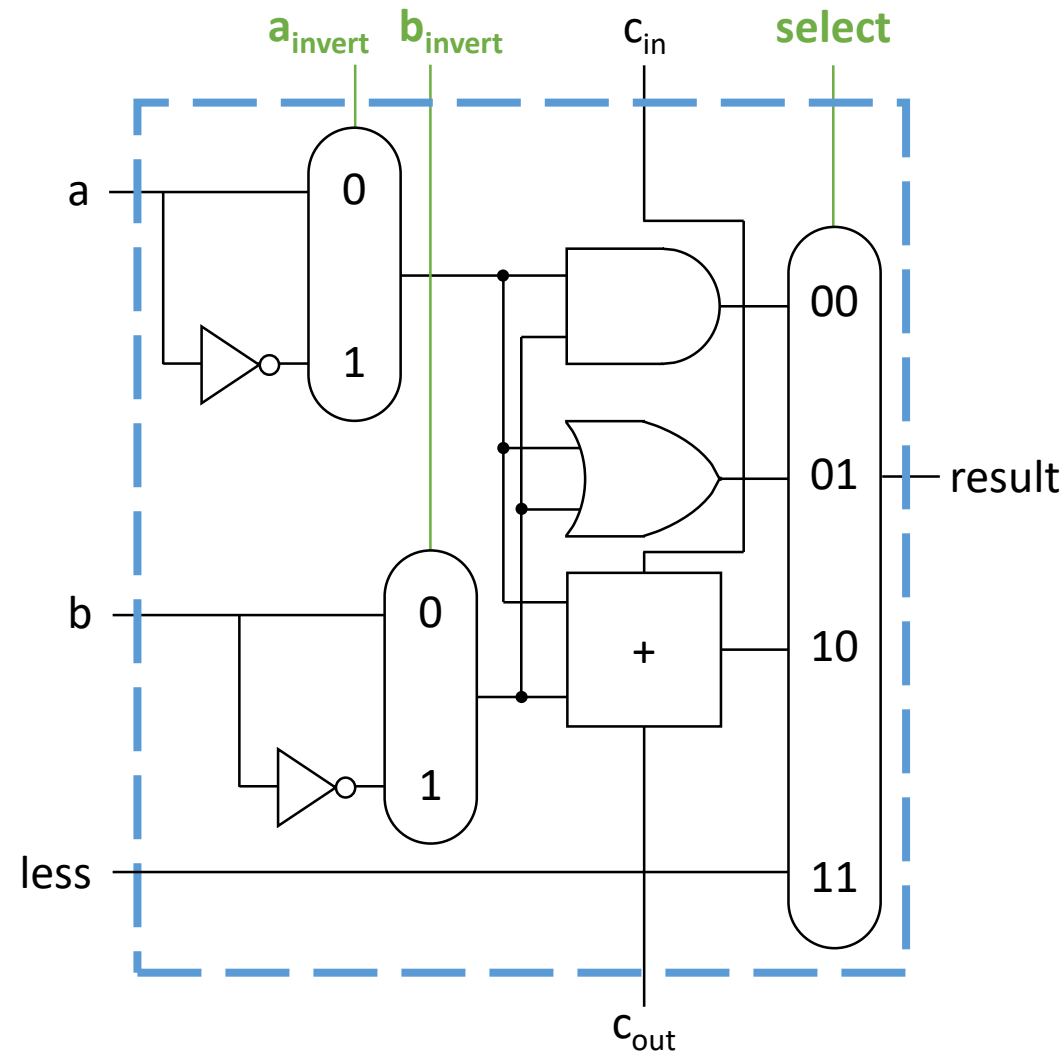
1-bit ALU for MSB with Overflow Detection



N-bit ALU with Overflow Detection



Values of Control Bits



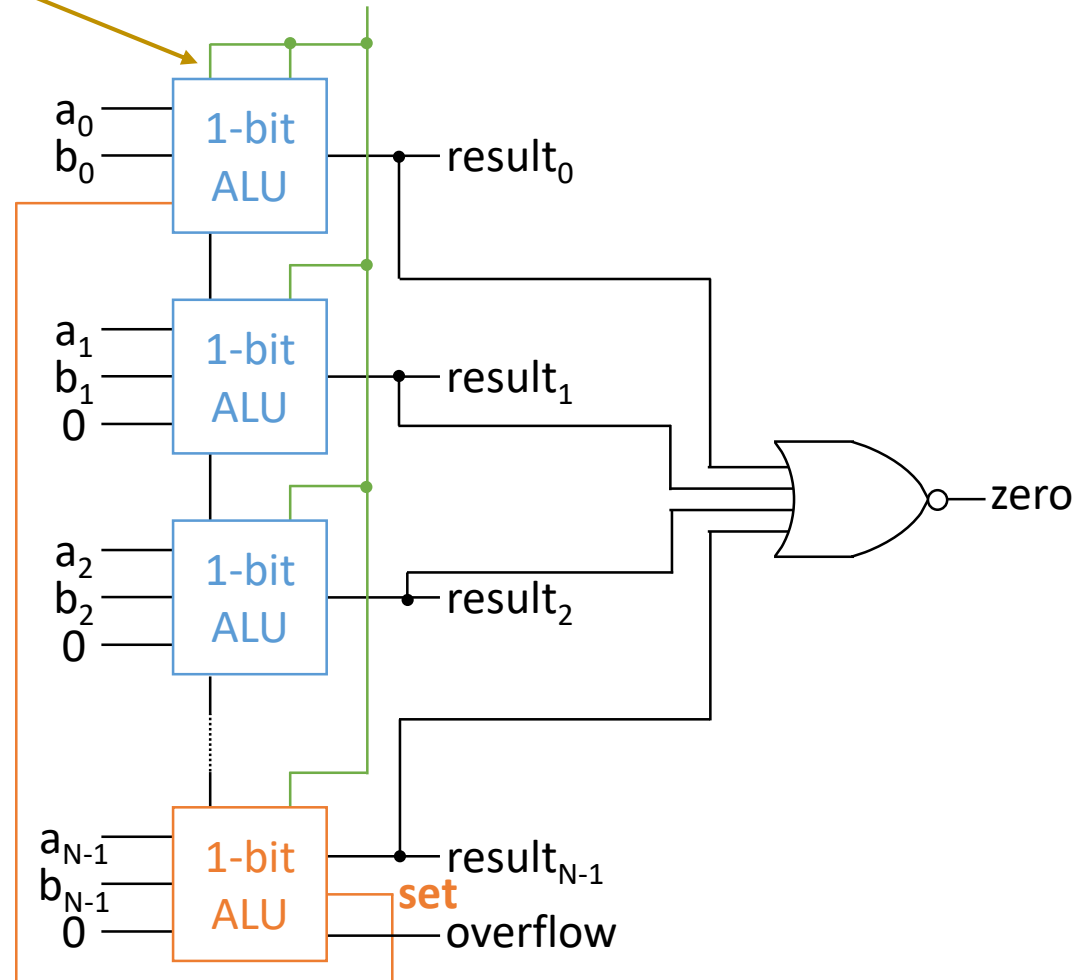
Operation	a _{invert}	b _{invert}	c _{in0}	select
AND	0	0	d	00
OR	0	0	d	01
NAND	1	1	d	01
NOR	1	1	d	00
ADD	0	0	0	10
SUB	0	1	1	10
SLT	0	1	1	11

c_{in0} is equivalent to b_{invert}

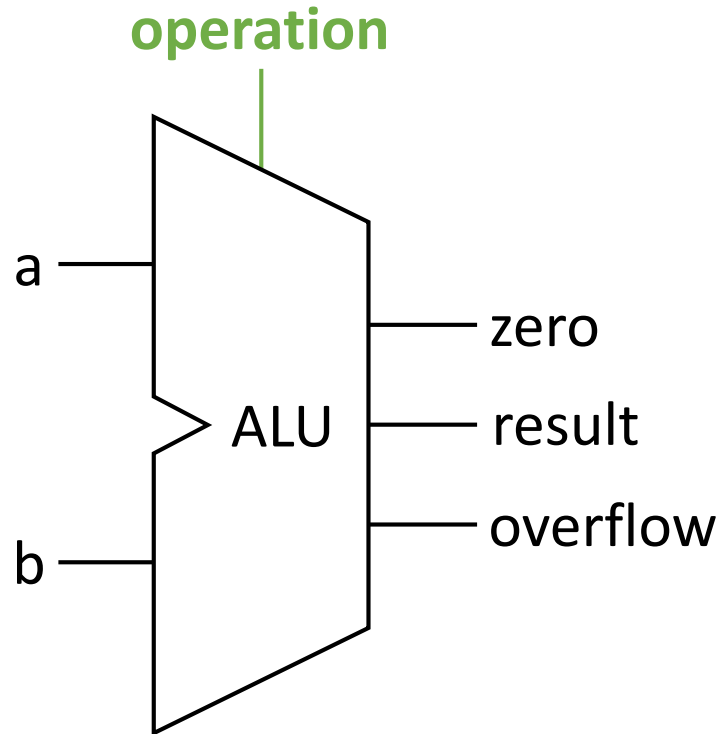
N-bit ALU

use b_{invert} as $c_{\text{in}0}$

operation (a_{invert} , b_{invert} , select)



N-bit ALU



Operation	Control Bits
AND	0000
OR	0001
NAND	1101
NOR	1100
ADD	0010
SUB	0110
SLT	0111

Textbook Sections

- Some of the content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - B.5